

Longest Common Subsequence

The longest common subsequence (LCS) is defined as the longest subsequence that is common to all the given sequences, provided that the elements of the subsequence are not required to occupy consecutive positions within the original sequences.

If s_1 and s_2 are the two given sequences then, z is the common subsequence of s_1 and s_2 if z is a subsequence of both s_1 and s_2 . Furthermore, z must be a **strictly increasing sequence** of the indices of both s_1 and s_2 .

In a strictly increasing sequence, the indices of the elements chosen from the original sequences must be in ascending order in z .

If

```
S1 = {B, C, D, A, A, C, D}
```

Then, $\{A, D, B\}$ cannot be a subsequence of s_1 as the order of the elements is not the same (ie. not strictly increasing sequence).

Let us understand LCS with an example.

If

```
S1 = {B, C, D, A, A, C, D}
```

```
S2 = {A, C, D, B, A, C}
```

Then, common subsequences are {B, C}, {C, D, A, C}, {D, A, C}, {A, A, C}, {A, C}, {C, D}, ...

Among these subsequences, {C, D, A, C} is the longest common subsequence. We are going to find this longest common subsequence using dynamic programming.

Using Dynamic Programming to find the LCS

Let us take two sequences:

X

A	C	A	D	B
---	---	---	---	---

The first sequence

Y

C	B	D	A
---	---	---	---

Second Sequence

The following steps are followed for finding the longest common subsequence.

1. Create a table of dimension $(n+1) \times (m+1)$ where n and m are the lengths of X and Y respectively. The first row and the first column are filled with zeros.

		C	B	D	A
	0	0	0	0	0
A	0				
C	0				
A	0				
D	0				
B	0				

2. Fill each cell of the table using the following logic.

- If the character corresponding to the current row and current column are matching, then fill the current cell by adding one to the diagonal element. Point an arrow to the diagonal cell.
- Else take the maximum value from the previous column and previous row element for filling the current cell. Point an arrow to the cell with maximum value. If they are equal, point to any of them.

		C	B	D	A
	0	0	0	0	0
A	0	0	0	0	1
C	0				
A	0				
D	0				
B	0				

Fill the values

		C	B	D	A
	0	0	0	0	0
A	0	0	0	0	1
C	0	1	1	1	1
A	0	1	1	1	2
D	0	1	1	2	2
B	0	1	2	2	2

- Step 2** is repeated until the table is filled.

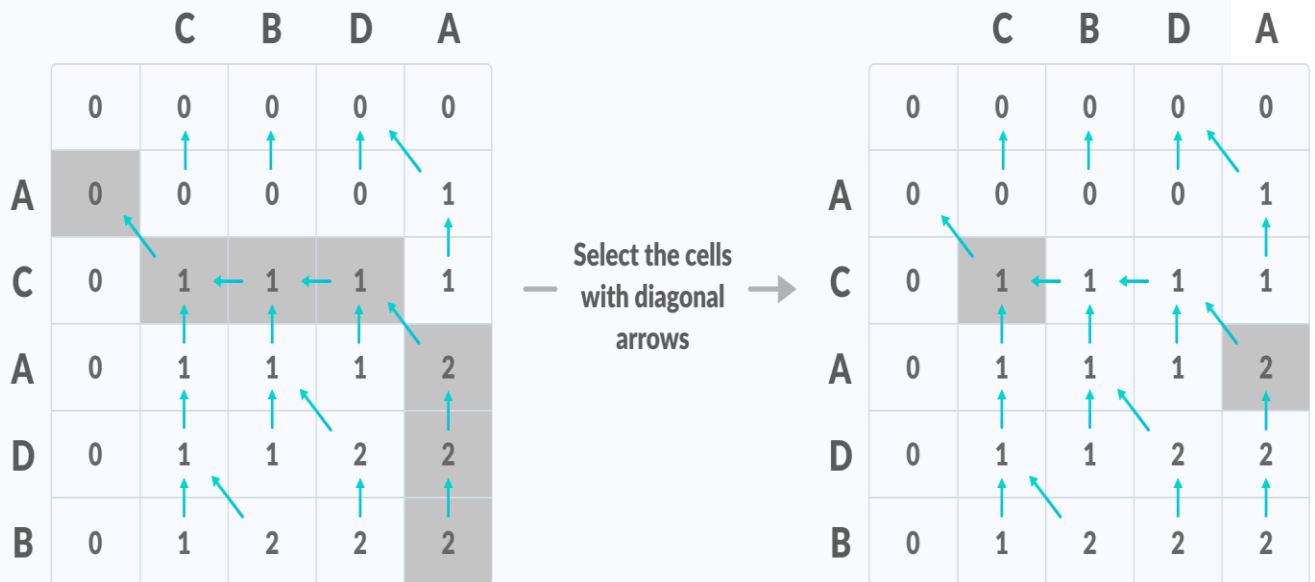
Fill all the values

- The value in the last row and the last column is the length of the longest common subsequence. The bottom right corner is the length of the LCS.

		C	B	D	A
A	0	0	0	0	0
C	0	0	0	0	1
A	0	1	1	1	2
D	0	1	1	2	2
B	0	1	2	2	2

7.

8. In order to find the longest common subsequence, start from the last element and follow the direction of the arrow. The elements corresponding to (↖) symbol form the longest common subsequence.



Create a path according to the arrows

Thus, the longest common subsequence is CD.

C A

How is a dynamic programming algorithm more efficient than the recursive algorithm while solving an LCS problem?

The method of dynamic programming reduces the number of function calls. It stores the result of each function call so that it can be used in future calls without the need for redundant calls.

In the above dynamic algorithm, the results obtained from each comparison between elements of x and the elements of y are stored in a table so that they can be used in future computations.

So, the time taken by a dynamic approach is the time taken to fill the table (ie. $O(mn)$). Whereas, the recursion algorithm has the complexity of $2^{\max(m, n)}$.